

2025-1

Basic Algorithm Study → For Beginners

기초 알고리즘 스터디 7주차

01

그리디

- 그리디하다는건 뭘까?

02

웰노운 그리디

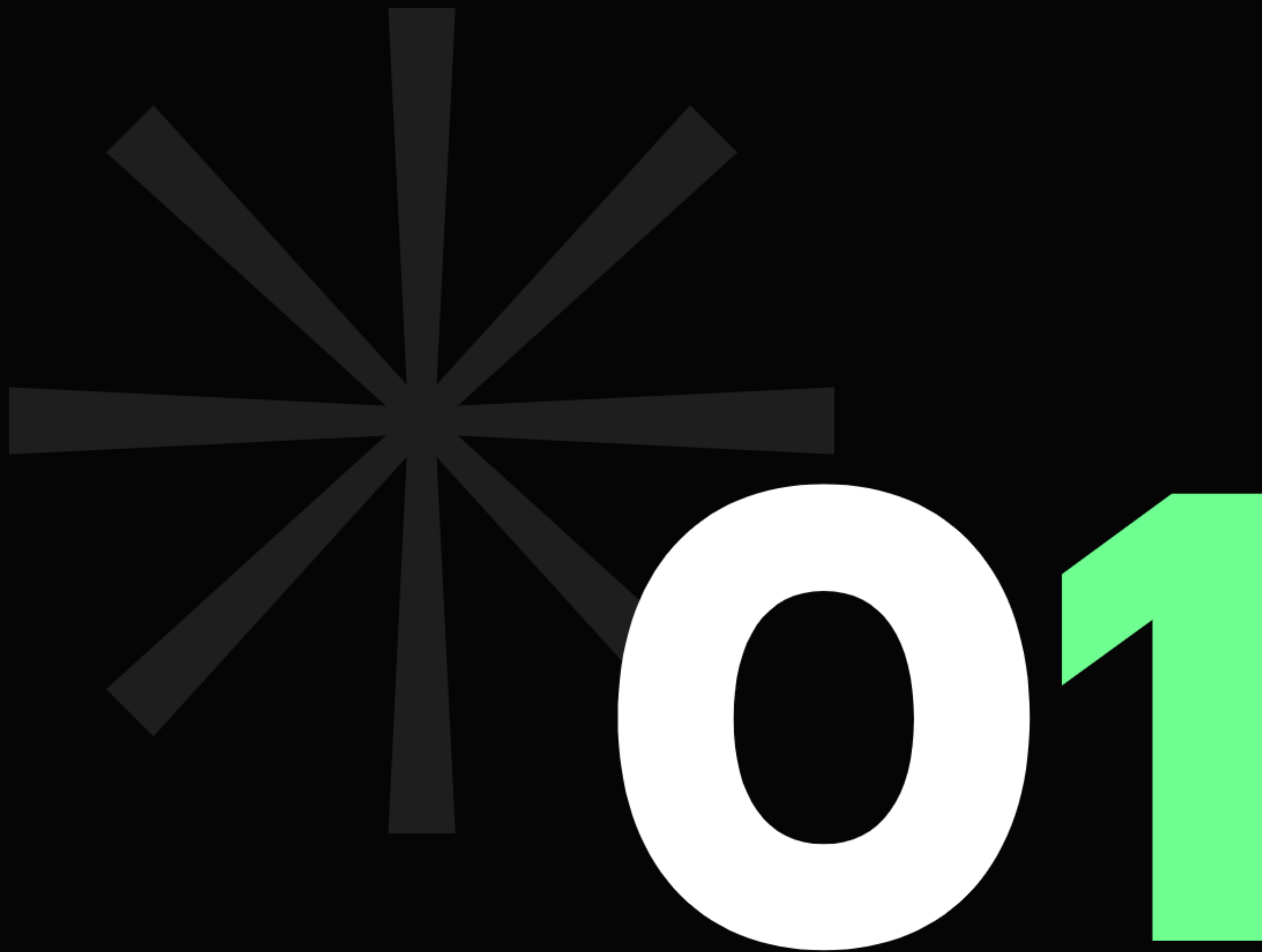
- 동전 0
- 보물
- 강의실 배정하기

03

문제 풀어보기

그리디

—





01

그리디하다는게 뭘까?

그리디 알고리즘(Greedy Algorithm)

탐욕법



01

그리디하다는게 뭘까?

탐욕적이라는 것은,
미래는 고려하지 않고
현재 상태에서 가장 좋아 보이는 선택을 한다는 뜻이다.



01

그리디하다는게 뭘까?

그때그때 최선의 선택을 하면
최선의 결과가 나올 수 있을까?



01

그리디하다는게 뭘까?

거스름돈을 주는 상황을 생각해 보자.
500원, 100원, 50원, 10원, 5원, 1원짜리 동전으로
정확히 N원을 만들면 된다.



01

그리디하다는게 뭘까?

이때, 가장 적은 수의 동전을 사용해서 N원을 맞추려면
어떻게 해야 할까?
가장 쉬운 방법이 하나 떠오를 것이다.



01

그리디하다는게 뭘까?

우선 N원에서 500원짜리로 줄 수 있는 만큼의 돈을 전부 주고,
남은 돈 중에서는 100원짜리로 줄 수 있는 만큼 전부 주고,
...(반복)

남은 돈은 1원짜리로 주면 되지 않을까?

01

그리디하다는게 뭘까?

예를 들어서 $N = 2764$ 라면

$$500*5 + 100*2 + 50*1 + 10*1 + 5*0 + 4$$

일 때

13개의 동전으로 거스름돈을 줄 수 있고,

이 값이 최적임을 증명할 수 있다.



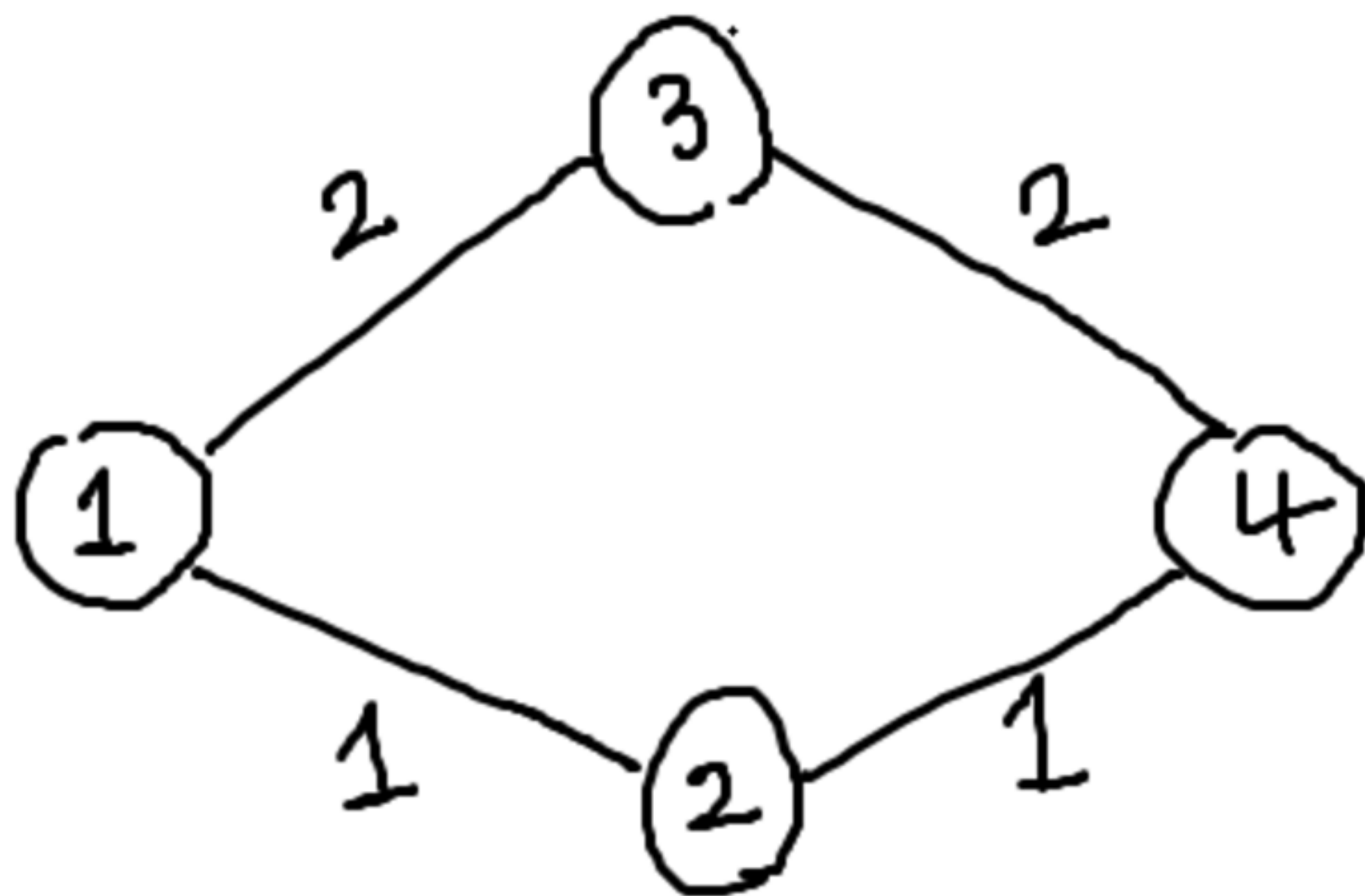
01

그리디하다는게 뭘까?

흠.. 그러면 이런 상황을 생각해 보자.
N개의 집이 집집마다 골목으로 이어져 있을 때
1번 집에서 N번 집으로 가는 **최단 경로**를 구해 보자.

01

그리디하다는게 뭘까?





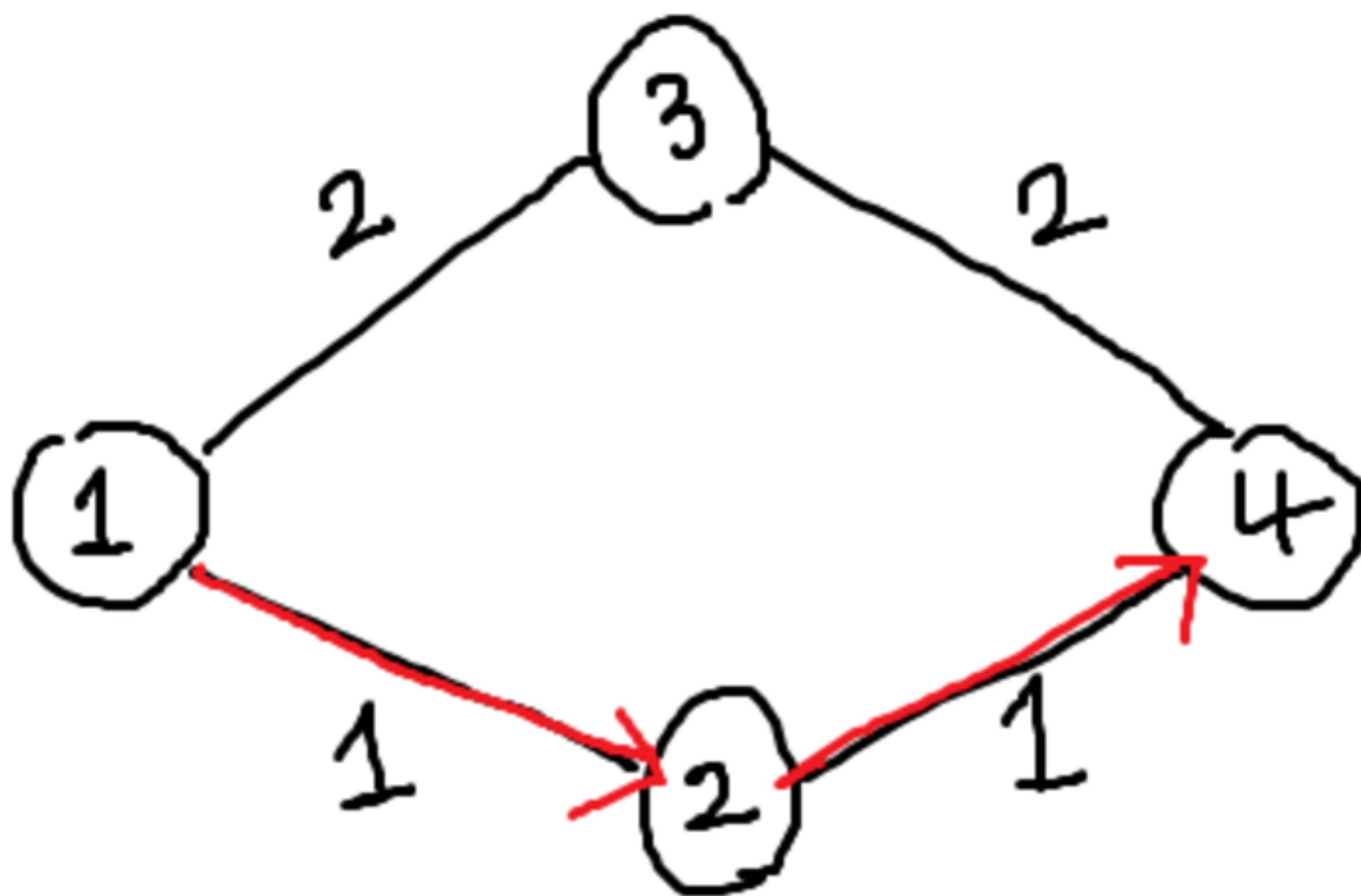
01

그리디하다는게 뭘까?

그리디한 방식으로
현재 갈 수 있는 골목 중 가장 짧은 골목만을 선택하면
최단 거리를 구할 수 있을까?
현재 상황에서는 그래 보이기도 하는데...

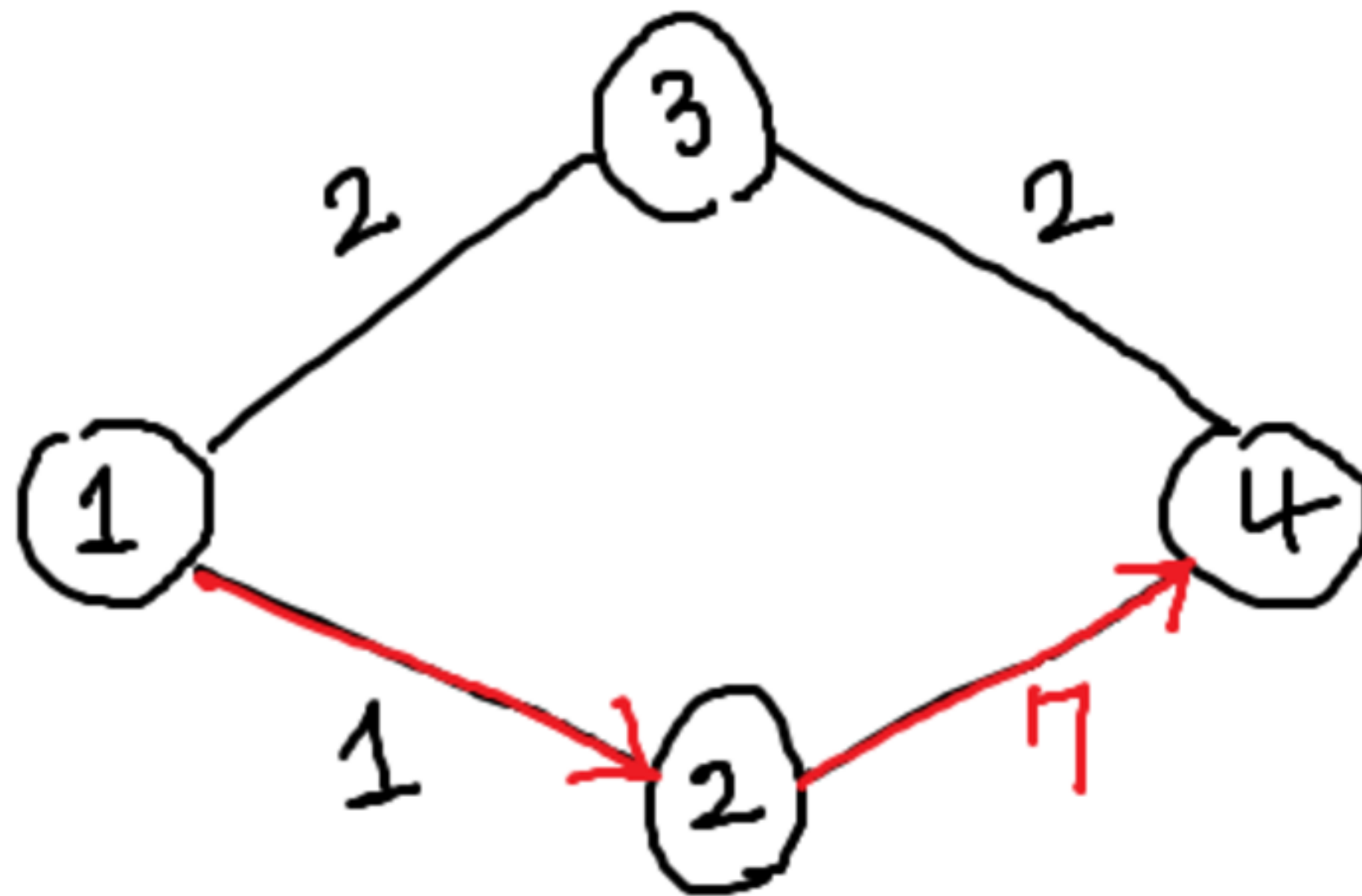
01

그리디하다는게 뭘까?



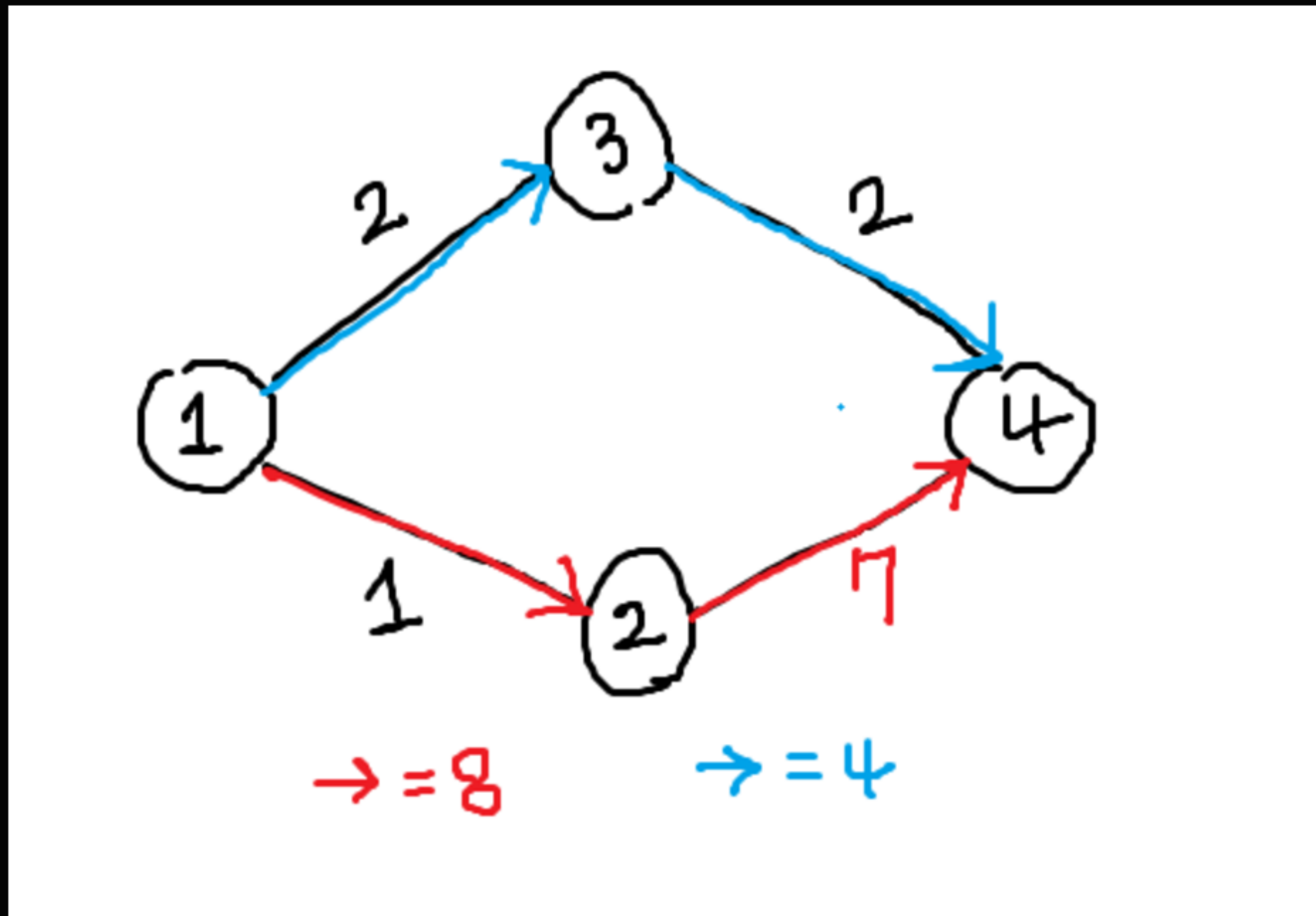
01

그리디하다는게 뭘까?



01

그리디하다는게 뭘까?





01

그리디하다는게 뭘까?

그리디한 해결법은

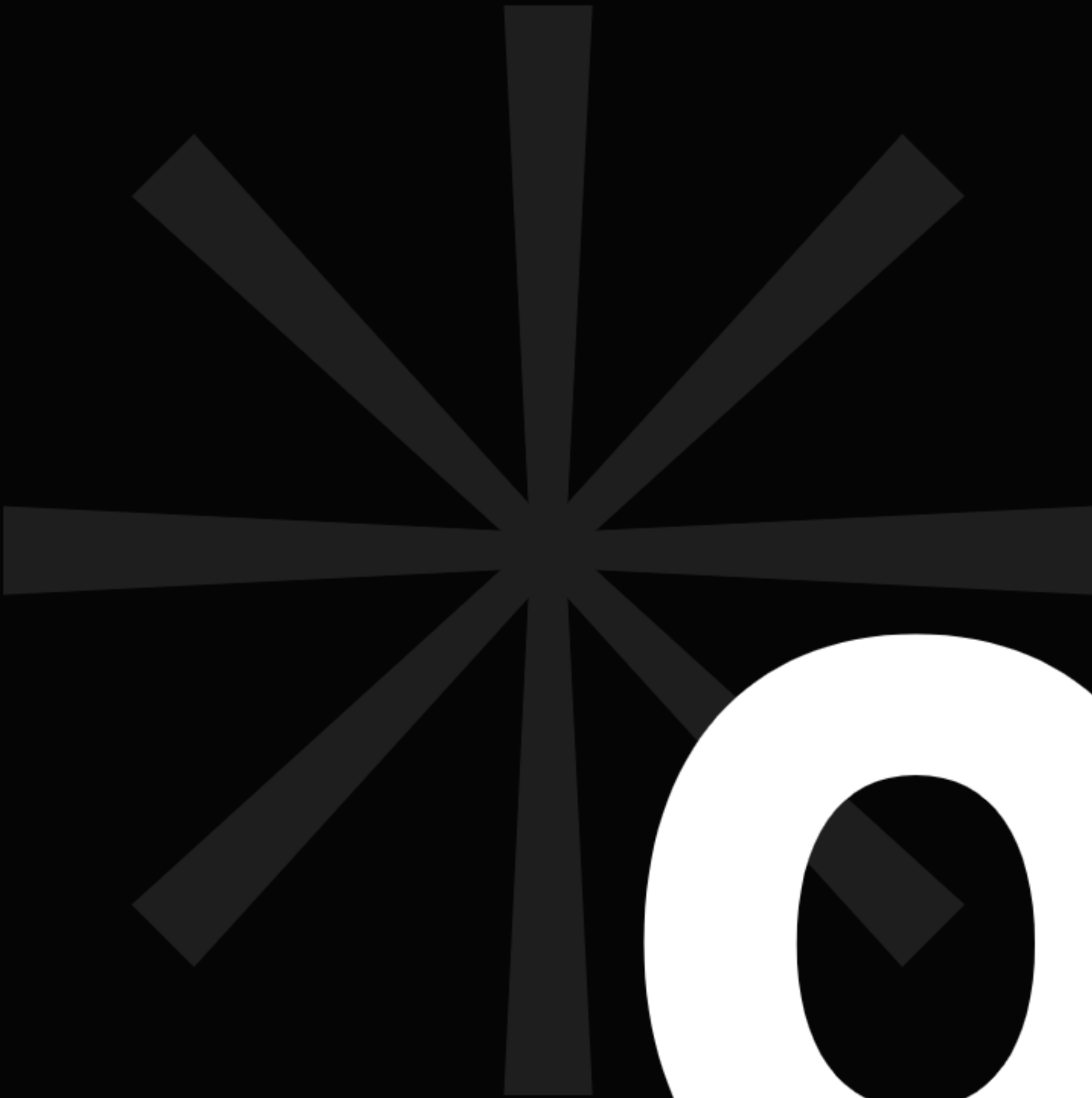
구현이 쉽고, 고려할 것이 많지 않아서 작동 속도가 빠르다는 장점이 있지만
항상 최적의 결과를 보장하지 않는다!

하지만 거스름돈처럼 **특정 상황**에서는 최적해를 얻을 수 있다.

따라서, 그리디 알고리즘을 적용하기 위해서는
탐욕적인 방식으로 최적해를 얻을 수 있는지 **증명**해야 한다.

웹노우
크리더

—

 02

간단한 그리디 문제를 풀면서 매번 증명을 하기는 쉽지 않다.

하지만 **그리디한 방식으로 풀 수 있다**는 것이 잘 알려진

웰노운(Well known) 문제들이 있다.

이 중에서 쉬운 몇가지 예를 소개하려고 한다.

다른 문제를 풀면서 비슷한 아이디어를 활용할 수 있는 경우가 나오면

아하~ 그거구나 그거 하면 된다.



02

월노운

예제: 동전 0(#11047)

아까 같이 살펴봤던 거스름돈 문제의 확장판이다.
K값의 범위가 매우 큰데, 어떻게 해야 빠르게 답을 찾아낼 수 있을까?

아까 500원, 100원... 을 나눠줬던 경험에 의하면
가장 큰 동전부터 줄 수 있는 만큼 전부 주면 될 것 같은데,
이 풀이를 증명할 수 있는 방법이 있을까?

문제의 조건에 주목하자.
 $i \geq 2$ 인 경우에 A_i 는 A_{i-1} 의 배수

당연한 사실이지만,
 A_i 는 $A_{i-2}, A_{i-3}, A_{i-4}, \dots A_1$ 의 배수이기도 하다.

증명:

A_{i-1} 은 A_{i-2} 의 배수이므로 $c^*(A_{i-2}) = A_{i-1}$

$$A_i = k^*(A_{i-1}) = k^*c^*(A_{i-2})$$

그럼 어떤 값 K 가 주어졌을 때,
 A_i 의 값을 가진 동전들로 K 를 나타냈다면

$$K = (A_i) * a + (A_{i-1}) * b + (A_{i-2}) * c + \dots + (A_1) * x$$

$$K = (A_i) * a + (A_{i-1}) * b + (A_{i-2}) * c + \dots + (A_1) * x$$

여기서 "그리디하지 않은" 방법으로
 $a+b+c+\dots+x$ 의 값을 그리디한 방법보다 작게 만들 수 있을까?

$$K = (A_i) * a + (A_{i-1}) * b + (A_{i-2}) * c + \dots + (A_1) * x$$

만약 그리디하지 않은 방법을 사용한다면,
어떤 상황에서 동전으로 바꿔줄 수 있는 양보다
더 적게 바꿔준 지점이 있을 것이다.

예를 들어서 400원이 남아있는 상황에서
100원짜리 4개를 바꾸지 않고 3개만 바꿨다든가...

$$K = (A_i) * a + (A_{i-1}) * b + (A_{i-2}) * c + \dots + (A_1) * x$$

이 지점을 대충 A_j 이라고 하자.

그럼 그리디하게 바꿨을 경우의 A_j 항을 $A_j * k$ 라고 했을 때

$A_j * w$ ($w < k$)로 바꿀 수 있을 것이다.

$$K = (A_i) * a + (A_{i-1}) * b + (A_{i-2}) * c + \dots + (A_1) * x$$

이 지점을 대충 A_j 이라고 하자.

그럼 그리디하게 바꿨을 경우의 A_j 항을 $A_j * k$ 라고 했을 때

$A_j * w$ ($w < k$)로 바꿀 수 있을 것이다.

그러면 $A_j(k-w)$ 만큼의 돈을 A_j 가 아닌 다른 동전으로 더 충당해야
K값을 맞출 수 있다.

$$K = (A_i) * a + (A_{i-1}) * b + (A_{i-2}) * c + \dots + (A_1) * x$$

여기서 문제가 생긴다.

우선 A_j 보다 더 작은 동전들로 값을 채우려면
당연하지만 $k-w$ 개의 동전보다 더 많은 동전을 사용해야 한다.

왜냐하면 동전들이 전부 배수 관계에 있기 때문이다.

만약 $A_j = A_k * c$ ($c > 1$)의 배율을 갖는 동전 A_k 로

$A_j(k-w)$ 만큼의 돈을 만들기 위해서는 $(k-w) * c$ 개의 동전을 사용해야 한다.

$$K = (A_i) * a + (A_{i-1}) * b + (A_{i-2}) * c + \dots + (A_1) * x$$

반대로 더 큰 동전을 이용할 수 있다면 전체 동전의 수는 줄어들 것이다.

결국에는 그리디한 방식으로 가야 최적해를 구할 수 있다.

하지만 이 증명은 동전들이 전부 배수 관계에 있을 때에만 성립한다.

그렇지 않은 경우의 반례는 쉽게 찾을 수 있다.

만약 10원, 7원, 1원으로 14원을 만들고자 한다면

$10*1 + 1*4$: 5개 보다 $7*2$: 2개가 더 최적의 결과임을 알 수 있다.

이럴 때는 다이나믹 프로그래밍을 활용해야 한다.

02

월노운

```
import sys
 = sys.stdin.readline

N, K = map(int, .split())
coins = []
for i in range(N):
    coins.append(int(.strip()))

res = 0
for i in reversed(coins):
    res += K//i
    K %= i

print(res)
```



02

월노운

예제: 보물(#1026)

이 문제도 직관적으로 생각해 보면
가장 큰 값에 가장 작은 값을 곱해야 할 것 같다.

그리고 실제로도 그것이 옳다.

만약 $A_i * B_i, A_j * B_j$ ($A_i > A_j, B_i > B_j$)의 순서로 매칭된 상태일 때

A_i 와 A_j 의 순서를 바꿔서 $A_j * B_i, A_i * B_j$ 로 바꾸면

$A_i * B_i + A_j * B_j > A_j * B_i + A_i * B_j$ 로 더 작아진다.

(A_i 대신 A_j+x , B_i 대신 B_j+y 로 바꿔서 계산해보면 쉽게 증명할 수 있다)

그럼 더 이상 같은 방식으로 교체할 값이 없는 상태를 생각해 보자.

B가 정렬된 상태일 때

A를 역순으로 정렬하면

$A_i * B_i, A_j * B_j$ ($A_i > A_j, B_i > B_j$)인 i, j 는 존재하지 않을 것이다.

문제에서는 B를 재배열(정렬)하지 말라고 했지만,
A는 순서를 마음대로 바꿀 수 있으므로
A를 적절히 재배열한 셈 치고 B를 정렬하더라도 상관이 없다.
어차피 재배열된 A는 정확히 구하지 않아도 된다.


```
N = int(input().strip())
A = list(map(int, input().split()))
B = list(map(int, input().split()))

A.sort()
B.sort(reverse=True)
res = [A[i]*B[i] for i in range(N)]

print(sum(res))
```




02

월노운



예제: 회의실 배정(#1931)



02

월노운



정렬을 하긴 해야 할 것 같은데,
시작하는 시간을 기준으로 정렬해볼까?

하지만 $(0, 3)$, $(1, 2)$, $(2, 3)$ 이 있다면
 $(0, 3)$ 보다 $((1, 2), (2, 3))$ 을 하는 게 더 이득이다.
이래서는 전혀 그리디한 해결책이 나오지 않을 것 같다.

반대로 생각해 보자.
이전 회의가 끝난 시각이 k 라면
다음 회의는 시작시간이 k 보다 느리거나 같은 회의만 열릴 수 있다.

그럼 최대한 많은 선택지를 가져가기 위해서는
끝나는 시간이 빠른 게 무조건 좋지 않을까?

끝나는 시간이 x 인 회의 A와, 끝나는 시간이 y 인 회의 B($x < y$)
y에서 선택할 수 있는 다음 회의들은
x에서도 전부 선택할 수 있다.

$x \sim y$ 사이에 열리는 회의들은 A에서만 선택할 수 있다.
따라서 A를 선택할 수 있다면 선택하는 것이 항상 옳다.

그렇다면 회의를 끝나는 시간 순서로 정렬한 뒤에 한개씩 돌면서 분석해보자.

만약 현재의 회의 종료 시간보다 시작 시간이 늦거나 같다면

해당 회의를 개최하고, 아니면 버린다.

회의를 개최할 경우 회의 종료 시간을 갱신한다.

하지만 신경써야 하는 부분이 있다.
바로 회의의 시작하는 시간과 끝나는 시간이 같을 수도 있다는 점이다.
(0, 3), (3, 3)의 회의 2개가 있다고 가정해보자.

실제로는 $(0, 3)$ 을 고른 뒤에 $(3, 3)$ 이 가능하지만
 $(3, 3)$ 을 먼저 선택한다면 회의의 종료 시간은 3이 되고,
 $(0, 3)$ 은 버려진다.

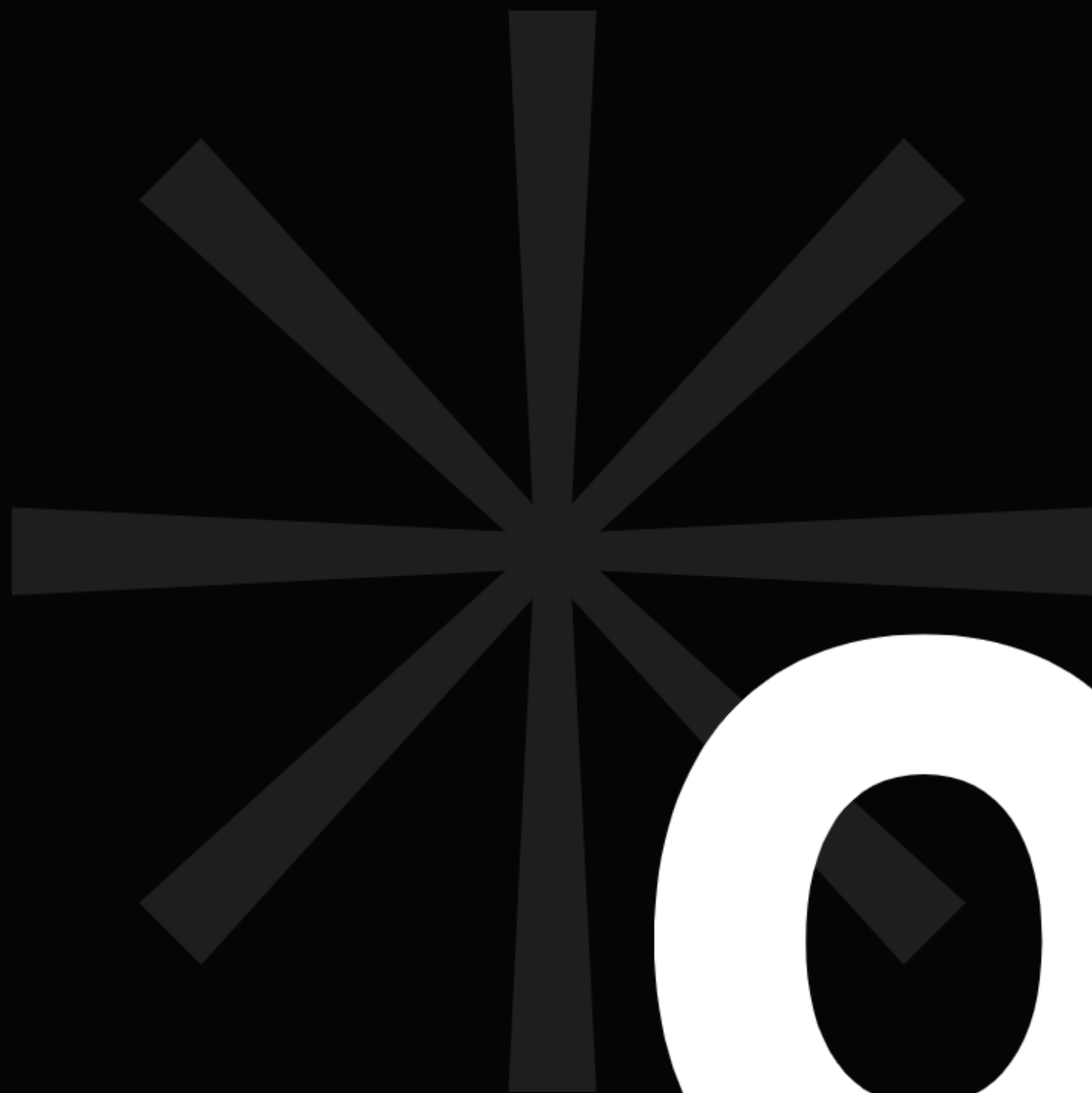
따라서 회의가 끝나는 순서대로 정렬하되,
끝나는 시간이 같다면 시작하는 시간이 앞서는 순서대로 정렬해야
그리디로 원하는 최적해를 얻을 수 있다.

```
N = int(input().strip())
table = []
for _ in range(N):
    table.append(tuple(map(int, input().split())))

table = sorted(table, key=lambda x: (x[1], x[0]))
temp = -1
cnt = 0
for st, ed in table:
    if st >= temp:
        cnt += 1
        temp = ed

print(cnt)
```


문제
풀어보기



03



03

문제 풀어보기

예제: 나는 기말고사형 인간이야(#23254)

만약 내가 천우라면?

당연히 공부 효율이 좋은 과목부터 공부해야

가장 많은 이득을 볼 수 있지 않을까?

다른 과목을 공부한다고 해서 특정 과목의 효율이 바뀌는 것은 아니기 때문이다.

정해진 시간 내에서 가장 많은 점수를 올려야 하니까
현재 상황에서 가장 많은 점수를 올릴 수 있는 과목을 공부해야 맞다.
그러나 최고점이 정해져 있으므로,
한계에 도달하면 과목의 공부 효율을 갱신해야 한다.

공부를 $N*24$ 번 진행해야 하니까
현재 최대 공부 효율을 $N*24$ 번 계산해야 하고,
갱신이 최대 M 번 일어날 수 있기에 매번 갱신하기에는
 $O(M*M)$ 의 시간이 걸린다.

하지만 우리는 이런 상황에 딱 맞는 자료구조를 알고 있다.
저번 시간에 배운 우선순위 큐를 활용해 보자.

우선 문제를 풀기 위해서는
각 과목의 효율이 저장된 힙에서 $N \cdot 24$ 번 $hq[0]$ 을 찾으면 된다.
 $O(N \cdot 24)$

그리고 $hq[0]$ 에 있는 과목이
다음 공부를 할 때 100점을 초과하게 된다면 효율을 갱신한다.
 $O(M \log M)$

그럼 이제 저장할 정보를 정해보자.

1. 어떤 과목의 **현재 점수**

2. 힙에 들어있는 **공부 효율과 해당 과목의 인덱스**

1은 한번 공부할 때마다 갱신하고,

2는 다음 공부에서 100점을 초과할 경우에만 갱신한다.

힙의 원소를 갱신하기 위해서는
한번 pop했다가 갱신한 값을 다시 push하면 된다.

```
import sys
from heapq import *
input = sys.stdin.readline

N, M = map(int, input().split())
score = list(map(int, input().split()))
ef = list(map(int, input().split()))
hq = []

for i in range(M):
    heappush(hq, (-min(100-score[i], ef[i]), i))

for i in range(N*24):
    d, idx = hq[0]
    score[idx] -= d
    if 100-score[idx] < -d:
        heappop(hq)
        heappush(hq, (-(100-score[idx]), idx))

print(sum(score))
```




강의 설문조사

QR 제공 예정

Thank you For Listening

7주차 스터디를 들어주셔서 감사합니다.
뒷풀이에 참여하실 분들만 강의실에 남아주세요!

—

